

Rivanna Runbook — Faithful 537 Sweep

These files run the faithful `537_Sports_Simulation` parameter sweep as Slurm job arrays, without adding any new mechanisms.

Files

- `faithful_537_sweep.py` — sweep grid and scenario runner.
- `faithful_537_sweep_rivanna_worker.py` — Rivanna worker (`stage1-shard` , `merge-stage1` , `stage2-shard` , `merge`).
- `rivanna_stage1_faithful_537.slurm` — **Stage 1 array** (default 64 tasks): shard CSVs under `stage1_shards/` .
- `rivanna_merge_stage1_faithful_537.slurm` — `merge-stage1` : concatenate shards → `stage1_results.csv` .
- `rivanna_stage2_array_faithful_537.slurm` — Stage 2 verification array (default 64 tasks).
- `rivanna_merge_faithful_537.slurm` — merge Stage 2 shards, grouped candidates, plots.

Single-process Stage 1 (no Slurm array):

`python faithful_537_sweep_rivanna_worker.py stage1 --reset` writes `stage1_results.csv` directly (debug / small tests).

Rivanna / Slurm Conventions

These scripts mirror the repo-level `pipe_job.slurm` conventions:

- `#SBATCH --account=sds_ag`
- `#SBATCH --mail-user=dzk3ja@virginia.edu`
- `#SBATCH --constraint=rivanna`
- `module load miniforge`
- `ENV_NAME="${ENV_NAME:-sports_net}"`
- line-buffered stdout/stderr via `stdbuf` when available

The scripts first try:

```
$HOME/.conda/envs/sports_net/bin/python
```

and fall back to `python` on PATH.

Submit From Repo Root

One file (like `pipe_job.slurm`)

From the **repository root** (same directory as `pipe_job.slurm`):

```
sbatch sim_job.slurm
```

That driver job submits **Stage 1 array** → **Merge Stage 1** → **Stage 2 array** → **final merge** and uses `sbatch --wait` on each step so one `sbatch` submission runs the full pipeline. Override shard counts if needed:

```
N_STAGE1_SHARDS=64 N_SHARDS=64 sbatch sim_job.slurm
```

The driver needs a long enough `#SBATCH --time` (default **12 hours**) because it stays active until all child jobs finish.

Manual dependency chain

On Rivanna, `cd` to the repository root (the directory containing `sports/`), then use a dependency chain so **Merge Stage 1** runs only after **all** Stage 1 array tasks finish:

```
j1=$(sbatch --parsable sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm)
j1m=$(sbatch --parsable --dependency=afterok:$j1 sports/outputs/simulation_sweeps/rivanna_merge_stage1_faithful_537.slurm)
j2=$(sbatch --parsable --dependency=afterok:$j1m sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm)
sbatch --dependency=afterok:$j2 sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
```

(`afterok` on the Stage 1 job id waits for the **entire** array to complete successfully.)

Outputs

All outputs land under:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/
```

Important outputs:

- `stage1_shards/stage1_shard_*.csv`
- `stage1_results.csv` (after `merge-stage1`)
- `stage2_shards/stage2_shard_*.csv`
- `stage2_results_merged.csv`
- `grouped_candidates.csv`
- `candidate_plots/`
- `README.md`

Shards

Stage 1 (default 64 tasks):

```
#SBATCH --array=0-63
N_STAGE1_SHARDS="${N_STAGE1_SHARDS:-64}"
```

If you change the array range, set `N_STAGE1_SHARDS` to match when submitting **Merge Stage 1**, e.g.:

```
#SBATCH --array=0-127
N_STAGE1_SHARDS=128 sbatch sports/outputs/simulation_sweeps/rivanna_merge_stage1_faithful_537.slurm
```

Stage 2 (default 64 tasks):

```
#SBATCH --array=0-63
N_SHARDS="{N_SHARDS:-64}"
```

If you change the Stage 2 array range, set `N_SHARDS` to match for **both** the Stage 2 array and the **final** merge script. Example for 128 shards:

```
#SBATCH --array=0-127
N_SHARDS=128 sbatch sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm
N_SHARDS=128 sbatch sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
```

Clean slate before a new `sim_job.slurm`

Sweep outputs and shard CSVs must not mix with an old run. From **repo root**:

```
./scripts/clean_rivanna_faithful_537_sweep.sh --dry-run # list what would be deleted
./scripts/clean_rivanna_faithful_537_sweep.sh --yes    # actually delete
```

This removes:

- `sports/outputs/simulation_sweeps/rivanna_faithful_537/` (entire tree)
- `slurm_out/slurm-537_*` and `slurm_out/slurm-sim_job-*` logs
- legacy `slurm-537_*` / `slurm-sim_job-*` in the repo root, if any

It does **not** delete other jobs' logs under `slurm_out/` (for example `pipe_job`). To wipe **all** Slurm logs:

```
./scripts/clear_slurm.sh
```

Optional: `./scripts/clean_rivanna_faithful_537_sweep.sh --yes --slurm-all` runs `clear_slurm.sh` after the sweep-specific deletes.

Fresh run checklist (Rivanna / Terminus + Mac)

Use this when you want a **full reset**, a new `sbatch sim_job.slurm`, and **results on your laptop**.

1. On Rivanna (SSH or Terminus on iPhone)

Working directory: **Ivy_Net repo root** (`cd` to the folder that contains `sports/`, `scripts/`, and `sim_job.slurm`).

Step	Action
1a	Optional: cancel bad or duplicate jobs — <code>squeue -u \$USER</code> , then <code>scancel <JOBID></code> if needed.
1b	Wipe prior sweep artifacts — <code>./scripts/clean_rivanna_faithful_537_sweep.sh --dry-run</code> then <code>./scripts/clean_rivanna_faithful_537_sweep.sh --yes</code> .
1c	Submit — <code>sbatch sim_job.slurm</code> (optional: <code>N_STAGE1_SHARDS=64 N_SHARDS=64 ENV_NAME=sports_net</code> on the same line).
1d	Monitor — <code>tail -f slurm_out/slurm-sim_job-<JOBID>.out</code> (job ID is printed when you submit).

2. Sync results to the Mac

On the Mac, from **repo root** (after jobs finish):

```
./scripts/rsync_pull_recent_hpc.sh quick
```

`quick` pulls `slurm_out/` plus sweep outputs under `sports/outputs/simulation_sweeps/` per the usual include/exclude rules. Use `./scripts/rsync_pull_recent_hpc.sh all` if you also need `tenure/tenure_pipeline`.

Narrow alternatives: `./scripts/rsync_pull_from_hpc.sh sweep` and/or `./scripts/rsync_pull_from_hpc.sh slurm_out`.

3. Git

Workflow cheat sheet: `docs/GIT_FOR_DUMMIES.md` (stash, pull/rebase, recovery). **Concepts:** `docs/GIT_MULTIPLE_MACHINES_ELEMENTARY.md`. **Doc index:** `docs/README.md`.

`rivanna_faithful_537/` and Slurm `slurm-*.out` / `slurm-*.err` are **gitignored** — you typically **do not** `git add` sweep CSVs or plots.

- **Commit / push** when you change **source**: Python worker, `*.slurm`, `sim_job.slurm`, docs, `faithful_537_sweep.py`, etc.
- After `git push`, on Rivanna run `git pull` in the same repo path so HPC matches Mac before the next `sbatch`.

Note: `./scripts/rsync_push_to_hpc.sh sweep` only syncs `sports/outputs/simulation_sweeps/` — not repo-root `sim_job.slurm`. If you edit `sim_job.slurm` on the Mac, use **git pull on Rivanna** or `scp` / `rsync` that file explicitly.

Where this doc lives

Markdown source: `sports/documents/RIVANNA_RUNBOOK.md`. Generated PDF (if present) is alongside it for reading on iPad/iPhone.

Not sports-specific: centralized guides (Git, cross-domain notes) live under `docs/` — start at `docs/README.md`.